

KM · Controle de Equipamentos

Aplicação interna de gestão de infraestrutura de TI: notebooks, acessórios e linhas telefônicas, com controle de usuários por papel (USER/ADMIN/MASTER), auditoria completa e dashboard de indicadores.

O projeto nasceu como uma migração do controle que era feito em planilha Excel para uma base de dados real (Supabase/PostgreSQL), com regras de negócio, permissões e histórico de movimentações garantidos no próprio banco — não só na tela.

Sumário da documentação

Este README cobre o essencial para rodar o projeto localmente. Para os detalhes:

Documento	Conteúdo
docs/arquitetura.md	Como o frontend é organizado: rotas, autenticação, papéis de usuário, camada de serviços, páginas
docs/banco-de-dados.md	Todas as tabelas, colunas, RLS, funções (RPC), triggers e o bucket de Storage
docs/mapeamento-banco-de-dados.md	Diagrama entidade-relacionamento e mapa completo de chaves estrangeiras entre as tabelas
docs/design-system.md	Identidade visual KM, tokens de cor, tema claro/escuro, componentes
docs/credenciais-e-seguranca.md	Quais credenciais o projeto usa, onde ficam guardadas e como trocá-las — leia antes de mexer em produção
docs/documentacao-funcional.md	O sistema explicado em linguagem simples, sem jargão técnico — papéis de usuário, telas, regras de negócio, glossário. Também disponível em Word: docs/docx/documentacao-funcional.docx
docs/especificacao-tecnica-ia.md	Especificação técnica consolidada de tudo (schema, RLS, RPCs, triggers, frontend, Edge Functions) — feita para servir de contexto/prompt a uma IA e de backup de conhecimento do projeto. Também em Word: docs/docx/especificacao-tecnica-ia.docx

Cada um dos 5 primeiros documentos também tem uma versão em PDF equivalente em [docs/pdf/](#), pra quem preferir ler fora do GitHub (ex.: compartilhar por e-mail). Elas são geradas a partir dos `.md` acima com `npm run docs:pdf` — rode esse comando de novo sempre que editar algum documento, **com o Microsoft Edge fechado** (o script usa o Edge em modo headless para imprimir o PDF, e uma janela do Edge já aberta intercepta a chamada e a

conversão falha). O único conteúdo que aparece diferente entre as duas versões é o diagrama Mermaid de [mapeamento-banco-de-dados.md](#): no GitHub ele é desenhado automaticamente; no PDF (estático) aparece como o código-fonte do diagrama.

Stack

- **Frontend:** React 19 + TypeScript + Vite, sem framework de UI (CSS próprio em [src/index.css](#))
- **Gráficos:** Recharts
- **Ícones:** [lucide-react](#)
- **Backend:** Supabase (PostgreSQL + Auth + Row Level Security + RPCs + Edge Functions + Storage)
- **Roteamento:** React Router 7
- **Exportação de PDF:** jsPDF + html2canvas
- **IA (leitura de faturas):** Google Gemini, chamado só pela Edge Function `process-invoice`

Pré-requisitos

- Node.js 20+
- Um projeto Supabase (pode ser um novo, vazio — a estrutura inteira é recriada pelos arquivos em `supabase/migrations/`)

Instalação e primeira execução

```
npm install
cp .env.example .env.local
# preencha VITE_SUPABASE_URL e VITE_SUPABASE_ANON_KEY em .env.local
# (veja docs/credenciais-e-seguranca.md pra saber onde pegar cada valor)
npm run dev
```

Se o projeto Supabase ainda estiver vazio (sem tabelas), rode a estrutura antes de usar o app: abra o SQL Editor do seu projeto Supabase e execute, nessa ordem, os arquivos de `supabase/migrations/` (ou o bundle já concatenado em `supabase/scripts/00_estrutura_completa_projeto_novo.sql`). Detalhes em [docs/banco-de-dados.md](#).

Scripts disponíveis

Comando	O que faz
<code>npm run dev</code>	Sobe o servidor de desenvolvimento (Vite)
<code>npm run build</code>	Type-check (<code>tsc -b</code>) + build de produção
<code>npm run preview</code>	Serve o build de produção localmente
<code>npm run lint</code>	Roda o linter (oxlint)
<code>npm run docs:pdf</code>	Regera os PDFs de <code>docs/pdf/</code> a partir dos <code>.md</code> (ver nota abaixo)
<code>npm run docs:docx</code>	Regera os Word (.docx) de <code>docs/docx/</code> a partir dos <code>.md</code>

Variáveis de ambiente

Variável	Onde é usada	Descrição
<code>VITE_SUPABASE_URL</code>	Frontend (<code>.env.local</code>)	URL do projeto Supabase
<code>VITE_SUPABASE_ANON_KEY</code>	Frontend (<code>.env.local</code>)	Chave pública (anon/publishable) do Supabase — segura para expor no browser
<code>SUPABASE_URL</code>	Scripts de migração (<code>.env.migration.local</code>)	Mesma URL, usada fora do browser
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Scripts de migração e Edge Functions	Chave com privilégio total — nunca vai para o frontend nem para o Git
<code>GEMINI_API_KEY</code>	Secret da Edge Function <code>process-invoice</code> (configurado no painel do Supabase, não em arquivo)	Chave da API do Google Gemini, usada só para ler faturas em PDF

Nenhum arquivo `.env*` (exceto `.env.example`, que não tem segredo nenhum) é versionado — veja o `.gitignore` e [docs/credenciais-e-seguranca.md](#).

Estrutura de pastas

src/	
components/	Componentes reutilizáveis (modais, cards, gráficos, branding)
contexts/	AuthContext (sessão/perfil) e ThemeContext (claro/escuro)
hooks/	useAuth, useTheme, useReferenceData
layouts/	AppLayout (sidebar + header) e AuthLayout (telas de login/cadastro)
lib/	Cliente Supabase e utilitário de exportação de PDF
pages/	Uma página por rota (Dashboard, Notebooks, Acessórios, Telefonia, ...)
routes/	Definição de rotas e guards (ProtectedRoute / PublicOnlyRoute)
services/	Uma função por operação de banco (notebooks.ts, employees.ts, ...) é a ÚNICA camada que fala com o Supabase; páginas não chamam o cliente Supabase diretamente
types/	Tipos TypeScript espelhando as tabelas do banco
supabase/	
migrations/	Uma migration por mudança de schema, em ordem cronológica – é a fonte da verdade da estrutura do banco
scripts/	Bundle de todas as migrations concatenadas (bootstrap rápido)
functions/	Edge Functions (admin-users, process-invoice)
scripts/migration/	ETL único (Excel → Supabase) usado na migração inicial dos dados não roda mais no dia a dia da aplicação
data/	Entrada/saída do ETL (arquivos reais não são versionados)
docs/	Documentação detalhada (ver sumário acima)
Imagens/	Artes e logos (identidade visual)

Papéis de usuário

Papel	Pode
USER	Ver dashboard, notebooks, acessórios, linhas telefônicas
ADMIN	Tudo do USER + cadastrar/editar ativos, colaboradores, departamentos, localidades, categorias, operadoras
MASTER	Tudo do ADMIN + aprovar/rejeitar/bloquear usuários, trocar papéis, ver auditoria completa, administrar contas (criar/editar/resetar senha/excluir), processar faturas por IA

Todo cadastro novo nasce com status **PENDING** e só usa o sistema depois de um MASTER aprovar — ver [docs/arquitetura.md](#).

Identidade visual

A marca é "KM": símbolo em estilo circuito (ciano/azul/violeta), tema escuro como padrão com um tema claro provisório. Detalhes completos, incluindo os tokens de cor e como adicionar o "White Mode" definitivo, em [docs/design-system.md](#).